Design Patterns

Design Pattern — proqramlaşdırmada **tez-tez rast gəlinən problemlərin sınaqdan çıxmış, yenidən istifadə edilə bilən həlləridir.**



Niyə istafədə olunur?

Məqsəd	İzah
🔁 **Təkrar istifadəyə imkan verir**	Eyni problemi hər dəfə sıfırdan yazmaq əvəzinə, hazır dizayn nümunəsini tətbiq edirsən.
🧠 **Daha asan anlaşılan struktur**	Digər proqramçılar bu patternləri tanıyır, kodunu tez başa düşür.
🛠️ **Kodun saxlanması və dəyişməsi asan olur**	Çünki patternlərdə məsuliyyətlər (responsibility) düzgün bölünür.
🧩 **Loose Coupling təmin edir**	Yəni obyektlər bir-birinə sıx bağlı olmur, kod daha çevik olur.

Üç əsas kateqoriya

1.Creational

2.Structural

3.Behavioral



- **CREATIONAL PATTERNS- obyektlərin yaradılma üsulunu idarə edir**

Pattern	İzah	Sadə Nümunə
Singleton	Sinifdən yalnız **bir obyekt** yaradıla bilər.	ConfigurationManager, Logger
Factory Method	Obyektin yaradılmasını alt siniflərə ötürür.	ShapeFactory.Create("circle")
Abstract Factory	Müxtəlif obyekt ailələri yaratmaq üçün.	UIFactory.CreateButton() – Windows və ya Mac üçün fərqli buton
Builder	Mürəkkəb obyektləri addım-addım qurur.	CarBuilder.BuildEngine().BuildWheels().Build()
Prototype	Mövcud obyektin **nüsxəsini** yaradır.	object.Clone()



STRUCTURAL PATTERNS-

obyektlər arasında münasibətləri qurur və strukturu səliqəli saxlayır

Pattern	İzah	Sadə Nümunə
Adapter	Uyğun gəlməyən interfeysləri uyğunlaşdırır.	AudioPlayer MP4 faylını oxumaq üçün adapter istifadə edir.
Bridge	İnterfeys və implementasiyanı ayırır.	Rəng və Şəkil siniflərini ayırmaq.
Composite	Hiyerarxik strukturu idarə edir.	Fayl sistemi: Folder → File
Decorator	Mövcud obyektə **yeni funksiya əlavə edir** (miras olmadan).	Coffee → WithMilkDecorator
Facade	Mürəkkəb sistemi **sadə interfeyslə** idarə edir.	HomeTheaterFacade.StartMovie()
Flyweight	Eyni məlumatı paylaşaraq yaddaşa qənaət edir.	Eyni simvolların paylaşılması (font rendering).
Proxy	Obyektə **dolayı giriş** təmin edir.	ImageProxy – şəkli yalnız tələb olunanda yükləyir.



BEHAVIORAL PATTERNS-

obyektlər arasında ünsiyyət və məsuliyyət bölgüsünü tənzimləyir

Pattern	İzah	Sadə Nümunə
Observer	Bir obyekt dəyişəndə digərləri xəbər tutur.	Event, Subscriber sistemi
Strategy	Davranışı dinamik dəyişmək üçün istifadə olunur.	PaymentStrategy – kart, PayPal və s.
Command	Əməliyyatı obyekt kimi saxlayır və sonradan işə salır.	Undo, Redo funksiyası
Mediator	Obyektlər arasındakı əlaqəni **mərkəzləşdirir**.	ChatRoom mediator
State	Obyektin **davranışı vəziyyətə görə dəyişir**.	Document – Draft, Published və s.
Iterator	Obyektin elementləri üzərində **ardıcıl keçidi təmin edir**.	foreach
Template Method	Əsas alqoritmi müəyyən edir, alt siniflər detallarını dəyişir.	AlgorithmBase.Execute()
Chain of Responsibility	Müraciəti ardıcıl handler-lərə ötürür.	SupportTicketHandler
Visitor	Yeni əməliyyatlar əlavə etməyə imkan verir.	DocumentVisitor – PDF, DOCX və s. üçün

DI ilə Service Locator arasındakı fərq nədir?

1. 🚫 Singleton gizli qlobal state yaradır

Yəni proqramın istənilən yerində **eyni instance** istifadə olunur.

Bu:

kodu debugging etməyi çətinləşdirir

yan-effektlərə səbəb olur

proqnozlaşdırmağı çətinləşdirir.

SİNGLETON

Məqsəd odur ki tək bir instans yaradıb proyekt boyunca ondan istifadə edək

Təsəvvür elə:
Səndə bir **loq sistemi (Logger)** var.
Proqramda harada new Logger() desən, hər dəfə yeni obyekt yaranacaq.
Amma biz istəmirik hər yerdə yeni logger açılsın — **bir dənə olsun və hamı ondan istifadə etsin.**
Yəni məqsəd:
Proqramda **yalnız bir dənə obyekt** yaransın və **hər yerdə eyni obyekt istifadə olunsun.**
#🟢 **Fikir:** Sanki ofisdə bir dənə printer var, hamı ona qoşulub çap eləyir.
❌ Əgər hər kəs öz printerini gətirsəydi — qarışıqlıq olardı.
Singleton bunu aradan qaldırır.

```
public class Logger
{
    private static Logger _instance; // tək obyekt saxlayırıq
    private Logger() {} // kənardan new etməyə icazə yoxdur

    public static Logger GetInstance()
    {
        if (_instance == null)
            _instance = new Logger(); // birinci dəfə yaradılır
        return _instance;
    }

    public void Log(string msg)
    {
        Console.WriteLine(msg);
    }
}
```

Niyə Singleton pattern çox zaman “anti-pattern” sayılır?

1. 🚫 **Singleton gizli qlobal state yaradır**

Yəni proqramın istənilən yerində **eyni instance** istifadə olunur.

Bu:

kodu debugging etməyi çətinləşdirir

yan-effektlərə səbəb olur

proqnozlaşdırmağı çətinləşdirir.

Global State nədir?

- **Global state** — proqramın hər yerindən çıxış edilə bilən və vəziyyəti dəyişdirilə bilən məlumatdır.
- Yəni:
- hər kəs görə bilir
- hər kəs dəyişə bilir
- dəyişiklik bütün sistemə təsir edir
- Bu, çox vaxt **nəzarətsiz paylaşılmış data** deməkdir.

- #**Global state** — proqramın hər tərəfindən görülə bilən və dəyişilə bilən datadır.
  Bu data dəyişəndə sistemin hər yerinə təsir edir, ona görə risklidir.

Niyə Singleton pattern çox zaman “anti-pattern” sayılır?

🚫 2. Test etmək çox çətindir

Unit testlərdə çox zaman hər test üçün **yeni obyekt lazımdır**.

Amma Singleton buna icazə vermir:

instansı reset etmək olmur

mock/ fake qoymaq çətinləşir

Bu səbəbdən testability-ni azaldır.

Niyə Singleton pattern çox zaman “anti-pattern” sayılır?

🚫 3. Single Responsibility Principle pozulur

Singleton həm:

obyekt yaradır

həm də state saxlayır

həm də global dependency rolunda çıxış edir

Yəni **bir neçə məsuliyyət** daşıyır → bu da OOP qaydasına ziddir.

Niyə Singleton pattern çox zaman “anti-pattern” sayılır?

🚫 4. Dependency Injection-ı pozur

Modern arxitekturalarda (ASP.NET Core və s.) hər şey **DI ilə idarə olunur**.

Singleton isə DI-dən kənarda işləyir və:

asılılıqları gizlədir

trace etmək çətindir

arxitekturanı pozur

Niyə Singleton pattern çox zaman “anti-pattern” sayılır?

🚫 5. Multi-threading problemleri

Yanlış implementasiya edilərsə:

iki müxtəlif thread eyni anda instance yarada bilər

race condition yarana bilər

thread-safe etmək üçün əlavə lock-lar lazımdır

MEDIATOR

Obyektlər arasındakı əlaqəni **mərkəzləşdirir**.

Mediator (Vasitəçi) — adından da göründüyü kimi, **obyektlər bir-biri ilə birbaşa danışmasın deyə araya vasitəçi salan** dizayn pattern-dir. Əgər sistemdə **çoxlu obyektlər bir-biri ilə əlaqə qurursa**, bu obyektlər arasında **qarışıqlıq və sıx bağlılıq (tight coupling)** yaranır.
Mediator bu əlaqəni **idarə edən tək bir vasitəçi** rolunu oynayır.

- Tutaq ki,biz istifadəçi var onlar bir birinə mesaj göndərir,Valeh,Əli,Aslan bir birinə mesaj göndərmək istəyirlər.

Valeh->Əli,

Əli->Aslan,

Aslan->Valeh ə və s istifadəçilərin sayı artıqca qarışıqlıq yaranacaq və obyektlər bir biri ilə birbaşa əlaqədə olacaq tight coupling yaranacaq. Məsələn 2 servisimiz var UserService və ShopService Shop da User i User də Shop istifadə etsək dependency yaranır bunun üçün Mediatordan istifadə edirik.

```csharp
public class UserService
{
    private readonly ShopService _shopService;

    public UserService(ShopService shopService)
    {
        _shopService = shopService;
    }

    public void BuyProduct(int userId, int productId)
    {
        _shopService.CreateOrder(userId, productId);
    }
}
```

```csharp
public class ShopService
{
    private readonly UserService _userService;

    public ShopService(UserService userService)
    {
        _userService = userService;
    }

    public void CreateOrder(int userId, int productId)
    {
        var user = _userService.GetUserById(userId);
        Console.WriteLine($"{user.Name} məhsul aldı!");
    }
}
```

•UserService → ShopService-dən asılıdır
•ShopService → UserService-dən asılıdır
Bu **dairəvi (circular dependency)** yaradır.
ASP.NET Core-da belə olduqda proqram **start zamanı belə xətaya düşür**:
“A circular dependency was detected for the service of type 'UserService'…”

Mediator lu yanaşma

```
public class BuyProductCommand : IRequest
{
    public int UserId { get; set; }
    public int ProductId { get; set; }
}

public class BuyProductHandler : IRequestHandler<BuyProductCommand>
{
    private readonly UserService _userService;
    private readonly ShopService _shopService;

    public BuyProductHandler(UserService userService, ShopService shopService)
    {
        _userService = userService;
        _shopService = shopService;
    }

    public async Task Handle(BuyProductCommand request, CancellationToken cancellationToken)
    {
        var user = _userService.GetUserById(request.UserId);
        await _shopService.CreateOrderAsync(user, request.ProductId);
    }
}
```

İndi UserService və ShopService bir-birindən xəbərsizdir,
Onlar Mediator üzərindən əlaqə saxlayırlar.
#“Əgər iki və ya daha çox servis bir-birindən istifadə edirsə, aralarında **circular dependency** yaranır.
Bu həm arxitekturanı pozur, həm də kodu test və saxlanılması çətin edir.
Mediator pattern bu problemi həll edir:
servislər birbaşa bir-birinə müraciət etmir,
əvəzində Mediator vasitəsilə bir-biri ilə əlaqə saxlayır.”

FACTORY

Factory pattern **obyektin yaradılma məntiqini mərkəzləşdirir**və **runtime şərtlərinə görə** hansı class-ın yaradılacağını qərar verir.

Tutaq ki, sənə **fərqli növ maşınlar** lazımdır: Car, Truck, Motorcycle.
Əgər hər yerdə new Car() və s. yazsan, kod qarışar.
Factory bunu həll edir: sən “fabrik”dən sadəcə “növ” istəyirsən, o isə uyğun obyekti verir.

#Bəs sual çıxa bilər ki DI varsa biz nə üçün Factory Pattern dən istifadə edirik,Factory Pattern runtime vaxtı qərar vermək olur,amma DI da olmur.

```
0 references
static void Main()
{
    IVehicle vehicle = VehicleFactory.CreateVehicle("car");
    vehicle.Drive();

}

4 references
public interface IVehicle
{
    3 references
    void Drive();
}

1 reference
public class Car : IVehicle
{
    2 references
    public void Drive() => Console.WriteLine("Car sürülür!");
}

1 reference
public class Truck : IVehicle
{
    2 references
    public void Drive() => Console.WriteLine("Truck gedir!");
}

1 reference
public class VehicleFactory
{
    1 reference
    public static IVehicle CreateVehicle(string type)
    {
        return type switch
        {
            "car" => new Car(),
            "truck" => new Truck(),
            _ => throw new ArgumentException("Yanlış növ!")
        };
    }
}
```

FACTORY METHOD

Bir sinifdə obyekt yaratmaq lazımdır, amma **hansı class-ın yaradılacağına qərarı alt siniflər versin**.

Tutaq ki,sənin belə bir kodun var:

```
var payment = new PayPalPayment();
```

Burda sən özün konkret olaraq deyirsənki,mənə PayPalPayment obyektini yarat!Yəni yaradılacaq obyektin növünü sən qərar vermisən.

Factory Method nə edir?

Biz obyektin yaradılma qərarını bir metoda veririk.Yəni artıq biz “new” demirik,metod özü qərar verir hansı obyektin yaradılacağına.

Sadə nümunə göstərək,tutalım mənim sistemimdə 2 ödəniş növü var:

- 1)PaypalPayment
- 2)StripePayment
- Və mən hər dəfə if-else yazmaq istəmirəm.

Şəkildəndə göründüyü kimi bizim ödəniş növümüz artsa əlavə if yazmalıyıq.Və bu zaman Factory Method Pattern köməyə gəlir.

```
string paymentType = "PayPal";

IPayment payment;

if (paymentType == "PayPal")
    payment = new PayPalPayment();
else
    payment = new StripePayment();

payment.Pay(100);
```

```
5 references
public interface IPayment
{
    3 references
    public void Pay(int amount);
}

1 reference
public class PayPalPayment : IPayment
{
    2 references
    public void Pay(int amount)
    {
        Console.WriteLine($"PayPal ilə {amount} AZN ödəndi.");
    }
}

1 reference
public class StripePayment : IPayment
{
    2 references
    public void Pay(int amount)
    {
        Console.WriteLine($"Stripe ilə {amount} AZN ödəndi.");
    }
}
```

```
public abstract class PaymentProcessor
{
    3 references
    public abstract IPayment CreatePayment();

    1 reference
    public void Process(int amount)
    {
        var payment = CreatePayment();
        payment.Pay(amount);
    }
}

1 reference
public class PayPalProcessor : PaymentProcessor
{
    2 references
    public override IPayment CreatePayment() => new PayPalPayment();
}

0 references
public class StripeProcessor : PaymentProcessor
{
    2 references
    public override IPayment CreatePayment() => new StripePayment();
}
```

Əvvəlcə interface lər yardılıb,miras verilib sonra isə Factory Method u olan abstract class yaradırıq.Və burdan göründüyü kimi artıq CreatePayment metodunun içərisində qərar verilir ki,hansı tipdə obyekt yaradılacaq

```
PaymentProcessor processor = new PayPalProcessor();
processor.Process(100);
```

ABSTRACT FACTORY

Abstract Factory — “Factory of Factories”-dir, yəni **bir neçə factory-nin idarə olunmasını** təmin edir.

```
5 references
public interface IButton { public void Render(); }
5 references
public interface IWindow { public void Render(); }


1 reference
public class WindowsButton : IButton { public void Render() => Console.WriteLine("Windows Button"); }
1 reference
public class WindowsWindow : IWindow { public void Render() => Console.WriteLine("Windows Window"); }


1 reference
public class MacButton : IButton { public void Render() => Console.WriteLine("Mac Button"); }
1 reference
public class MacWindow : IWindow { public void Render() => Console.WriteLine("Mac Window"); }


3 references
public interface ComplexFactory
{
    3 references
    public IButton CreateButton();
    3 references
    public IWindow CreateWindow();
}

0 references
public class WindowsFactory : ComplexFactory
{
    2 references
    public IButton CreateButton() => new WindowsButton();
    2 references
    public IWindow CreateWindow() => new WindowsWindow();
}

1 reference
public class MacFactory : ComplexFactory
{
    2 references
    public IButton CreateButton() => new MacButton();
    2 references
    public IWindow CreateWindow() => new MacWindow();
}
```

```
ComplexFactory Mac = new MacFactory();
Mac.CreateWindow().Render();
Mac.CreateButton().Render();
```

•**WindowsFactory** və **MacFactory** — bunlar sadəcə “zavodlardır”.
•**Abstract Factory (IGUIFactory)** isə bu zavodların **ümumi idarəetmə qaydasıdır**.

Ona görə deyirik:
“Abstract Factory” — “Factory of Factories”-dir, çünki o, **birdən çox zavodun (factory-nin)** idarəçiliyini təmin edir.

STRATEGY PATTERN

Strategy Pattern — **bir funksiyanın (alqoritmin)** bir neçə variantını (strategiyasını) yaradır və **hansı strategiyanın işləyəcəyinə runtime-da qərar verir.**

Strategy Pattern

Strategy Pattern — bir davranış (loqika) üçün **bir neçə fərqli alqoritm** (strategiya) varsa və biz **hansı alqoritmdən istifadə ediləcəyini runtime-da (iş vaxtı)** seçmək istəyiriksə, bu pattern istifadə olunur.
Yəni davranışı (məntiqi) dəyişmək istədiyimizdə if/else, switch ilə kodu çirkləndirməyək, əvəzində hər davranışı ayrıca **"strategiya"** kimi saxlayıb, **istədiyimizi inject** edək.

Məsələn,bir e-commerce saytında 3 ödəniş üsulu var:

- CardPayment
- CashPayment

Amma ödənişin ümumi məqsədi **"ödəməni yerinə yetirmək"**dir.

```
0 references
public class PaymentService
{
    0 references
    public void MakePayment(string paymentType)
    {
        if (paymentType =="Card")
        {
            Console.WriteLine("Paid with Credit Card");
        }
        else
        {
            Console.WriteLine("Paid with Cash");
        }
    }
}
```

```
void Console.WriteLine(string? value) (+ 17 overloads)
Writes the specified string value, followed by the current line terminator, t
```

```
public interface IPaymentStrategy
{
    3 references
    public void Pay(int amount);
}

0 references
public class Cash : IPaymentStrategy
{
    2 references
    public void Pay(int amount)
    {
        Console.WriteLine($"{amount}Azn Nağd Ödəniş edildi.") ;
    }
}

1 reference
public class Card : IPaymentStrategy
{
    2 references
    public void Pay(int amount)
    {
        Console.WriteLine($"{amount}Azn Kartla Ödəniş edildi.");

    }
}


2 references
public class PaymentSpace
{
    IPaymentStrategy _payment;
    1 reference
    public PaymentSpace(IPaymentStrategy payment)
    {
        _payment = payment;
    }

    1 reference
    public void DoPayment(int amount) {

        _payment.Pay(amount);
    }
}
```

ADAPTER PATTERN

Adapter Pattern - Uyğun gəlməyən interfeysləri uyğunlaşdırır.

Təsəvvür et ki, sən Azərbaycandan Amerikaya səyahətə gedirsən. Telefon şarj cihazını götürürsən, amma Amerikada rozetkalar fərqlidir! Sənin şarj cihazının ştəbəkəsi Azərbaycan rozetkasına uyğundur, Amerika rozetkasına deyil. Nə edirsən? **Adapter** alırsan! Bu kiçik cihaz bir tərəfdən Azərbaycan şəbəkəsini qəbul edir, digər tərəfdən Amerika rozetkasına uyğunlaşır.

Proqramlaşdırmada Adapter Pattern də eyni məntiqdir.

Adapter Pattern nədir?

Adapter Pattern bir **dizayn patterndir** (design pattern) və belə bir problemi həll edir:
İki fərqli interfeys (sistem) var, amma bir-biri ilə işləyə bilmir. Adapter onları bir-birinə uyğunlaşdırır.
Yəni:

- Sənin **mövcud bir kodun** var
- Yeni bir **kitabxana/sistem** istifadə etmək istəyirsən
- Amma onlar **fərqli dillə danışır** (fərqli metodlar, fərqli strukturlar)
- **Adapter** bu ikisini bir-birinə bağlayır

Niyə belə bir şey var?

Problem 1: Köhnə kod + Yeni sistem
Təsəvvür et ki, şirkətdə 5 il əvvəl yazılmış bir sistem var. İndi yeni bir ödəniş sistemi əlavə etmək istəyirsən, amma:
•Köhnə kod makePayment() metodu istifadə edir
•Yeni sistem isə processTransaction() metodu istifadə edir
Bütün köhnə kodu dəyişmək çox risklidir və vaxt aparır. **Adapter yaradırsan** ki, köhnə kod dəyişməsin, amma yeni sistemlə işləsin.
Problem 2: Üçüncü tərəf kitabxanaları
YouTube API-dən video məlumatları alırsan. Sonra Vimeo-ya keçmək qərara alırsan. Amma:
•YouTube: getVideoData()
•Vimeo: fetchVideoInfo()
Tətbiqdəki bütün yerləri dəyişmək əvəzinə, **Adapter yazırsan**.
Problem 3: Test etmək
Real verilənlər bazası ilə test etmək təhlükəlidir. **Adapter** ilə fake/mock verilənlər bazası yaradırsan, amma kodun dəyişmi

Əsas üstünlükləri

1. Köhnə kod dəyişmir (Open/Closed Principle)

Mövcud sisteminə toxunmursan, sadəcə yeni təbəqə əlavə edirsən.

2. Asanlıqla genişləndirilir

Yeni format əlavə etmək istəyirsən? Sadəcə yeni adapter yaz.

3. Single Responsibility

Hər adapter yalnız bir işlə məşğuldur: uyğunlaşdırma.

4. Test etmək asandır

Real sistemləri fake adapterlərlə əvəz edə bilərsən.